C++ Day 1

Founder of C++

- The founder of the **C++ programming language** is **Bjarne Stroustrup**.
- He developed C++ at **Bell Labs** in the **early 1980s** as an **extension of the C programming language**.
- Initially, it was called **"C with Classes"**.

Creating and Running a C++ Project in Visual Studio

Step 1: Create a New Project

1. Go to **File → New → Project**
2. **Select:** *Empty Project*
3. Click **Next**
4. **Project Name:** project1
5. **Location:** C:\MyCpp
6. **Select:** *Place solution and project in the same directory*
7. Click the **"Create"** button

Step 2: Add a New Source File

1. Right-click on **"project1"** in *Solution Explorer*
2. Select **Add → New Item**
3. Choose **C++ File (.cpp)**
4. Name it **First.cpp**
5. Click **Add**

Step 3: Write the Program

File: First.cpp

```
#include <iostream>
using namespace std;
int main()
{
   cout << "Hello World!" << endl;
}
```

Step 4: Compile the Program

1. Go to **Build → Compile** or press **[Ctrl + F7]**
2. Check the folder:
   C:\MyCpp\project1\x64\Debug
   You will find the file:
   project1.obj — *generated by the compiler*

Step 5: Build the Solution

1. Go to **Build → Build Solution** or press **[Ctrl + Shift + B]**

2. Check the folder:
   C:\MyCpp\project1\x64\Debug
   You will find the file:
   project1.exe — *generated by the linker*

Step 6: Run the Application

Press **[Ctrl + F5]** to run the application.

Difference Between \n and endl

Both **\n** and **endl** cause a **new line** to be printed.
However, there is a key difference between the two.

- **endl**
  - Inserts a newline character and **flushes the output buffer**.
  - Hence, endl is **more reliable** than \n.

Example Using endl

```
cout << "hello" << endl;
```

Explanation:

1. "hello" is inserted into the output buffer first.
2. endl inserts a **newline character (\n)** and **flushes** the output buffer.
3. The **flush operation** ensures that "hello" and the newline \n are **immediately written** to the console.

Example Using \n

```
cout << "hello\n";
```

Explanation:

- Without endl, the buffered data (like "hello") **could remain in the output buffer** for a while.
- This depends on the **platform** and **system behavior**.
- The data may not be displayed immediately — it might be flushed:
  - When the **program ends**,
  - When the **buffer becomes full**, or
  - During **other output operations**.

This is why **output buffering** can sometimes cause **delays** in displaying text.

C/C++ Build Process

When you write and run a C or C++ program, several stages occur before you see the final output.

---

1. Preprocessing

- Handles directives such as #include, #define, #ifdef, etc.
- Produces an **expanded source file** (.i) with all macros and headers replaced.

---

2. Compilation

- Converts the preprocessed .i file into **assembly code** (.asm).
- Checks for **syntax errors**.
- The generated assembly code is **platform-specific** and depends on the CPU architecture.

---

3. Assembling

- Translates assembly code into **machine code**, producing an **object file** (.obj or .o).
- The output is **not executable** yet.
- The object file contains:
    - **Code section:** machine instructions
    - **Data section:** initialized global/static variables
    - **BSS section:** uninitialized global/static variables
    - **Symbol table:** information about variables and functions

---

4. Linking

- Combines **object files** and **library code** (like printf() or standard library functions).
- Resolves **external references** — functions or variables declared but defined elsewhere.
- Adjusts **relative addresses** to form a consistent memory layout.
- Produces the final **executable file** (.exe on Windows or a.out on Linux).

What is Linking and Why is it Important?

After your .cpp or .c code is compiled, you get a .obj file (object file), but it's **not ready to run yet**.

The **.obj file is incomplete** because:

- It only contains the machine code for your functions.

- Your code often calls other functions (e.g., printf(), scanf(), or display() from another file).
- These functions are not inside your .obj, so the **linker's job** is to find them, combine them, and connect them properly.

What Does the Linker Do?

Finds missing pieces

- Looks for functions or global variables your code uses but hasn't defined.
- These may come from:
    - Standard libraries (C/C++ standard library)
    - Other .obj files you wrote

Combines object files

- Merges code and data sections from multiple .obj files into one.
- Example: if main.cpp calls display() from utils.cpp, the linker merges both .obj files.

Resolves symbol addresses

- Each .obj file contains **relative addresses**, not actual memory locations.

Example:

int x = 10, y = 20;

The compiler assigns them as:

- x at offset 0
- y at offset 4 (assuming int is 4 bytes)

If another file has:

int z = 30;

It will also assign z at offset 0.

When the linker combines these .obj files, it adjusts:

- x → 0
- y → 4
- z → 8

This process assigns each variable a **unique position** in the final executable.
It is called **relocation**, which means changing relative (offset) addresses into a combined layout.

Why Are Offsets Important?

Offsets make .obj files:

- **Independent** — multiple files can have offset 0 or 4 for their variables.
- **Relocatable** — the linker adjusts them during the final build.

This modularity allows:

- Writing large projects with multiple source files.
- Reusing compiled libraries.
- Efficient compilation (only recompile changed files).

---

5. Loading

- The operating system loads the .exe into memory when it is executed.
- No address change is required — everything is already **position-independent**.
- Execution begins from the **main()** function.

Executable Formats

- **Windows:** uses .exe (PE format – Portable Executable)
- **Linux:** uses **ELF** (Executable and Linkable Format)

Why does this matter?
Different operating systems expect a specific binary format:

- Windows loader understands only **PE**
- Linux loader understands only **ELF**

Therefore, .obj or .exe files are **not portable** between operating systems.
This makes **C/C++ platform-dependent languages**.

Can a C/C++ Program Be "Compile Once, Run Anywhere" Within the Same OS?

Yes, but under specific conditions:

- The **compiler** and **system architecture** (e.g., 32-bit or 64-bit) must be the same.
- If transferring **object files (.obj)**, the compiler version must also match.
- If transferring the **final executable (.exe or Linux binary)**, it will run as long as:
    - The same **operating system** is used (Windows or Linux), and
    - The required **libraries** are available.

Example:
A C++ file compiled on one Windows PC can run on another Windows PC **without recompiling**, provided both systems have similar setups.

Typedef

The typedef keyword is used to create an **alias name** for a datatype.

Syntax:

typedef <datatype> <aliasname>;**Example:**

Instead of writing:

```
long unsigned int num = 10;
```

You can now write:

```
ul num = 10;
```

Structure Example:

```
struct EmployeeDetails
{
  char ecode[20];
};

struct EmployeeDetails e1;
```

Or using typedef:

```
typedef struct EmployeeDetails
{
  char ecode[20];
} emp;

emp e1;
```

Pointer

A **pointer** is a variable that holds the **address of another variable**.

Syntax:

datatype *pointervariable;

Examples:

```
int* ptr1;   // pointer to int
char* ptr2;  // pointer to char
float* ptr3; // pointer to float
```

Any type of pointer occupies **8 bytes** in a **64-bit memory address space**.

Pointer Operators

- & → Address of
- * → Value at the address contained by the pointer

Example:

```
int num1 = 20;
int* ptr1;
ptr1 = &num1;

printf("%d\n", num1);
printf("%d\n", *ptr1);
```

Accept Two Numbers and Print Using Pointers

```
void main()
{
  int num1, num2, * ptr1, * ptr2;
  puts("Enter two numbers");
  scanf("%d%d", &num1, &num2);
  ptr1 = &num1;
  ptr2 = &num2;

  printf("%d\t%d\n", *ptr1, *ptr2);
  printf("%d\n", *ptr1 + *ptr2);
  printf("%d\n", *ptr1 - *ptr2);
  printf("%d\n", *ptr1 * *ptr2);
  printf("%d\n", *ptr1 / *ptr2);
  printf("%d\n", *ptr1 % *ptr2);
```

```
}
```

Swapping Two Numbers Using Pointers

```
void main()
{
  int num1, num2, temp, * ptr1, * ptr2;
  puts("Enter 2 numbers");
  scanf("%d%d", &num1, &num2);
  ptr1 = &num1;
  ptr2 = &num2;

  printf("Before Swapping %d\t%d\n", *ptr1, *ptr2);

  temp = *ptr1;
  *ptr1 = *ptr2;
  *ptr2 = temp;

  printf("After Swapping %d\t%d\n", *ptr1, *ptr2);
}
```

Pointer Arithmetic

Pointers can be **incremented** or **decremented** only.

Formula:

pointervariable ± (specified number × scale factor)

Scale factor = size of the datatype the pointer refers to.

Example:

```
int num1 = 50;
int* ptr1 = &num1;

printf("%d\n", *ptr1);
ptr1++;  // pointer arithmetic
```

Pointer arithmetic is mainly useful in **arrays**.

Two pointers can be subtracted only if they are of the **same type**.

Formula:

(first pointer - second pointer) / scale factor

Pointer and Functions

Call by Value

```
void main()
{
  int num1 = 5, num2 = 8;
  void swap(int, int);
  printf("Before %d\t%d\n", num1, num2);
  swap(num1, num2);
  printf("After %d\t%d\n", num1, num2);
}

void swap(int x, int y)
{
  int temp = x;
  x = y;
  y = temp;
}
```

Call by Address

```
void main()
{
  int num1 = 5, num2 = 8;
  void swap(int*, int*);
  printf("Before %d\t%d\n", num1, num2);
  swap(&num1, &num2);
  printf("After %d\t%d\n", num1, num2);
}

void swap(int* x, int* y)
{
  int temp = *x;
  *x = *y;
  *y = temp;
}
```

Call by Address and Call by Value

```
void main()
{
  int num1 = -5, num2 = -8;
  void change(int, int*);
  printf("Before %d\t%d\n", num1, num2);
  change(num1, &num2);
  printf("After %d\t%d\n", num1, num2);
}

void change(int x, int* y)
```

```
{
  x *= -1;
  *y *= -1;
}
```

Pointer to Function

Pointer pointing to a function.

Syntax:

returntype(*pointervariable)(arguments);

Examples:

```
int (*ptr)(int);          // accepts int, returns int
void (*ptr)(int, char);   // accepts int, char, returns nothing
int (*ptr)();             // accepts nothing, returns int
void (*ptr)(int, int, float);// accepts int, int, float, returns nothing
void (*ptr)();            // accepts nothing, returns nothing
```

Example Program:

```
void main()
{
  int sqr(int);
  int num = 5, res;
  int (*ptr)(int);
  ptr = sqr;

  res = sqr(num);
  // or
  res = ptr(num);
  // or
  res = (*ptr)(num);

  printf("%d\n", res);
}

int sqr(int k)
{
  return k * k;
}
```

Pointer and Arrays

```
void main()
{
   int arr[4];
   int i;

   for (i = 0; i < 4; i++)
   {
      scanf_s("%d", &arr[i]);
      // or
      scanf_s("%d", arr + i);
   }

   for (i = 0; i < 4; i++)
   {
      printf("%d\n", arr[i]);
      // or
      printf("%d\n", *(arr + i));
   }
}
```

Array name is also a kind of **pointer**.

```
int arr[4] = { 10, 20, 30, 40 }, * ptr, i;

ptr = arr;
// or
ptr = &arr[0];

for (i = 0; i < 4; i++)
{
   printf("%d\n", arr[i]);
   *(arr + i);
   ptr[i];
   *(ptr + i);
}

while (ptr < arr + 4)
{
   printf("%d\n", *ptr++);
}
ptr = arr;
```

Character Pointer Example

```
int i;
char str[] = "hello";
char* ptr = str;
// or
char* ptr = &str[0];
```

Character by character printing:

```
for (i = 0; str[i] != '\0'; i++)
printf("%c\n", str[i]);

for (i = 0; ptr[i] != '\0'; i++)
printf("%c\n", ptr[i]);
```

String printing:

```
puts(str);
puts(ptr);

ptr++;
puts(ptr);
ptr--;
```

Using pointer increment/decrement:

```
while (*ptr)
printf("%c\n", *ptr++);

ptr = str;
printf("%c\n", *ptr);
printf("%c\n", *ptr++);
printf("%c\n", *ptr);
printf("%c\n", ++ * ptr);
printf("%c\n", *ptr);
printf("%c\n", (*ptr)--);

puts(str);
puts(ptr);
```

Function Accepting an Array

```
void main()
{
  int arr[4] = { 10, 20, 30, 40 };
  void disp(int*);

  disp(arr);
```

```
    // or
    disp(&arr[0]);
}

void disp(int* ptr)
{
    int i = 0;

    for (i = 0; i < 4; i++)
        printf("%d\n", ptr[i]);

    i = 0;
    while (i < 4)
    {
        printf("%d\n", *ptr++);
        i++;
    }
}
```

Function Returning an Array

Example 1 (Invalid – returns local array):

```
int* fun()
{
    int arr[4] = { 10, 20, 30, 40 };
    return arr;
}
```

Example 2 (Valid – returns static array):

```
int* fun()
{
    static int arr[4] = { 10, 20, 30, 40 };
    return arr;
}
```

Calling Function:

```
void main()
{
    int* fun();
    int* ptr, i;
    ptr = fun();

    for (i = 0; i < 4; i++)
        printf("%d\n", ptr[i]);
}
```

Function to Toggle Case of a String

```
void toggle(char*);

void main()
{
   char str[30];
   puts("Enter your name");
   gets_s(str);

   printf("Before %s\n", str);
   toggle(str);
   printf("After %s\n", str);
}

void toggle(char* ptr)
{
   int i;
   for (i = 0; ptr[i] != '\0'; i++)
   {
      if (ptr[i] >= 65 && ptr[i] <= 90)
         ptr[i] += 32;
      else if (ptr[i] >= 97 && ptr[i] <= 122)
         ptr[i] -= 32;
   }
}
```

Array of Pointers

An array that contains addresses.

```
int a = 10, b = 20, c = 30;
int* arr[] = { &a, &b, &c };

for (int i = 0; i < 3; i++)
{
   cout << *arr[i] << endl;
}
```

Double Pointer

A pointer that points to another pointer.

```
int num1 = 300;
int* ptr1 = &num1;
int** ptr2 = &ptr1;

cout << num1 << "\t" << *ptr1 << "\t" << **ptr2 << endl;
```

Dynamic Memory Allocation (DMA)

DMA means **allocating memory during runtime**.

The header file **malloc.h** provides four functions:

- malloc() — allocates new memory
- calloc() — allocates and initializes memory
- realloc() — reallocates existing memory
- free() — deallocates memory

Example 1: Integer Input Using malloc

```
#include <iostream>
using namespace std;

void main()
{
  int* ptr, i, rec;
  puts("How many numbers");
  scanf_s("%d", &rec);

  ptr = (int*)malloc(rec * sizeof(int));

  printf("Enter %d values\n", rec);
  for (i = 0; i < rec; i++)
    scanf_s("%d", &ptr[i]);

  puts("Displaying");
  for (i = 0; i < rec; i++)
    printf("%d\n", ptr[i]);

  free(ptr);
  puts("Done");
}
```

Example 2: String Input Without Array

```
#include <iostream>
using namespace std;

void main()
{
  char* ptr;
  int rec, i;
```

```
    cout << "How many characters" << endl;
    cin >> rec;

    ptr = (char*)malloc((rec + 1) * sizeof(char));

    for (i = 0; i < rec; i++)
        cin >> ptr[i];

    ptr[i] = '\0';

    cout << ptr << endl;
    free(ptr);
    cout << "Done" << endl;
}
```

C Preprocessor Directives

Introduction

- Before a C program is compiled, the **source code** is first processed by a program called the **preprocessor**.
- This step is known as **preprocessing**.
- Commands used by the preprocessor are called **preprocessor directives**, and they always begin with the symbol **#**.

List of Common Preprocessor Directives

No.	Directive	Syntax / Keywords	Description
1	**Macro Definition**	#define name value	Defines a symbolic name (macro) for a constant value or expression. The compiler replaces every occurrence of the name with the defined value during preprocessing.
2	**Header File Inclusion**	#include <file_name> or #include "file_name"	Inserts the contents of another file into the program before compilation. < > is used for standard library files, and " " is used for user-defined files.
3	**Conditional Compilation**	#if, #ifdef, #ifndef, #else, #elif, #endif	Controls which parts of code are compiled. Used to include or exclude sections depending on whether certain macros or conditions are defined.
4	**Undefine and Compiler Instructions**	#undef, #pragma	#undef removes a previously defined macro. #pragma gives special instructions to the compiler, such as memory alignment or executing functions before and after main().

Example 1: General Use of Preprocessor Directives

```
#include <stdio.h>      // Header file inclusion

// 1. Macro Definition
#define PI 3.14         // Defines a constant value
#define SQUARE(x) (x * x) // Defines a macro function

// 2. Conditional Compilation
#define DEBUG           // Enables debug-related code

// 3. Other Directives
#undef PI               // Removes the definition of PI
#pragma startup init    // Runs before main()
#pragma exit cleanup    // Runs after main()

void init() {
   printf("Program starting...\n");
}

void cleanup() {
```

```
    printf("Program ending...\n");
}

int main() {
    printf("Square of %d = %d\n", 4, SQUARE(4));  // Macro example

#ifdef DEBUG
    printf("Debug mode is ON\n");   // Executes only if DEBUG is defined
#else
    printf("Debug mode is OFF\n");
#endif

#ifndef PI
    printf("PI is undefined now\n");  // True because of #undef
#endif

    return 0;
}
```

Example 2: Use of #pragma pack(1)

```
#include <iostream>
using namespace std;

#pragma pack(1)   // Tells the compiler to pack structure members with 1-byte alignment

struct mystruct {
    char ch = 'A';
    double d = 6.7;
} m;

int main() {
    cout << m.ch << "\t" << m.d << endl;
    cout << sizeof(m) << endl;   // Prints total size of structure
}
```

Explanation of #pragma pack(1)

Concept	Explanation
Purpose	Controls how structure members are aligned in memory.
Default Behavior	The compiler adds padding between members to align data (for faster access).
With #pragma pack(1)	Padding is disabled — members are stored tightly without gaps.
Result	Structure size becomes smaller (more memory-efficient), but access speed may reduce slightly.

Example of Alignment Difference

Setting	Memory Behavior	Structure Size (for char + double)
Default (no pragma)	Members aligned to natural boundaries (padding added)	16 bytes (1 byte + 7 padding + 8 bytes)
With #pragma pack(1)	Members packed tightly (no padding)	9 bytes (1 + 8)

Summary

- #pragma pack(1) → Removes padding, minimizes structure size.
- Default alignment → Adds padding for performance.
- Use #pragma pack(1) when **memory optimization** is more important than **access speed**.

Practical Demonstration: Viewing Preprocessed Output

To **view the preprocessed file** generated by the compiler:

1. **Right-click** on your project in Visual Studio.
2. Go to
3. Properties → Configuration Properties → C/C++ → Preprocessor
4. Set the option **"Preprocess to a file"** to **Yes**.
5. **Compile** the project again.
6. Navigate to the following directory:
7. D:\MyCpp\ConsoleApplication1\x64\Debug
8. You will find a file named **ConsoleApplication1.i**, which contains the **preprocessed source code**.

Setting Command-Line Arguments in Visual Studio

When you want to pass **command-line arguments** to your C or C++ program in Visual Studio, follow the steps below:

Steps:

1. **Right-click** on the project name in **Solution Explorer**.
2. Select **"Properties"** from the context menu.
3. Navigate to the following path:
4. Configuration Properties → Debugging → Command Arguments
5. Click on the **"Edit"** button next to **Command Arguments**.
6. **Type your arguments** in the edit box.
    - Separate multiple arguments using **spaces**.
    - Example:

    - arg1 arg2 arg3
7. Click **Apply**, then **OK** to save the settings.

Purpose:

- These arguments will be automatically passed to the program **whenever it runs from Visual Studio**.
- This simulates running the program with arguments from the command line, e.g.:
- program.exe arg1 arg2 arg3

Storage Classes in C/C++

Storage classes in C/C++ define **how and where variables are stored**, their **default initial values**, **scope**, and **lifetime**.
They directly influence memory management and variable accessibility across the program.

---

1. Memory Segmentation in C/C++ Programs

When a C/C++ program executes, memory is divided into multiple segments, each serving a specific role:

Text Segment

- **Contains:** Compiled machine code (the actual instructions of the program).
- **Property:** Usually **read-only** to prevent accidental modification.

Data Segment

- **Contains:** Initialized **global** and **static** variables.
- **Subdivided into:**
    - **Initialized Data Segment:** For variables explicitly initialized (e.g., int x = 10;)
    - **Uninitialized Data Segment (BSS):** For global/static variables declared but not initialized (e.g., int y;)
- **BSS Segment:** Automatically **zero-initialized** by the system during runtime.

Heap

- **Used for:** Dynamically allocated variables (malloc, calloc, new, etc.).
- **Lifetime:** Persists until explicitly released using free or delete.

Stack

- **Used for:** Local (automatic) variables and function call management (return addresses).
- **Lifetime:** Exists during function execution; released once function returns.
- **Behavior:** Follows **Last-In, First-Out (LIFO)** order.

Summary of Memory Areas

Segment	Purpose
Text Segment	Compiled program instructions
Data Segment	Initialized global/static variables
BSS Segment	Uninitialized global/static variables
Heap	Dynamic memory allocation
Stack	Local variables and function calls

2. BSS vs. Data Segment

Feature	BSS Segment	Data Segment
Content	Uninitialized or zero-initialized global/static variables	Initialized global/static variables
Example	int x;	int y = 10;
Initialization	Automatically zero-initialized at runtime	Initialized with given values before runtime
File Size	Does not occupy space in executable	Occupies space (stores initial values)
Memory Allocation	Occurs during program load	Occurs during program load

Do Variables Move Between Segments at Runtime?

No.
If a variable starts in the **BSS segment** (uninitialized static/global), it **stays there** even if assigned a value later at runtime.
The **segment classification is fixed at compile/link time**, not runtime.

Program Execution Flow (Variable Allocation Journey)

The following example shows how a global variable cnt travels from source code to runtime memory:

```
// first.c
extern int cnt;     // Declaration (no memory)
void fun() {
   puts("in fun");
   printf("%d\n", cnt);
}

// second.c
#include "first.c"
int cnt = 100;      // Definition (memory allocated later)
void main() {
   printf("in main %d\n", cnt);
   fun();
}
```

(1) Compile Time

Each .c file is compiled separately into an .obj file.

File	Status
first.c	extern int cnt; → Declaration only (undefined symbol)
second.c	int cnt = 100; → Definition (symbol defined)

- No memory allocation yet — only symbolic references exist.

---

(2) Link Time

The linker combines .obj files and resolves symbol references.

- Finds extern cnt in first.obj
- Matches it with cnt definition in second.obj
- Assigns a **virtual (logical) address**, e.g., 0x601050

Still, no physical memory is allocated yet.

---

(3) Load Time

The **OS loader** loads the executable into memory.

Segment	Content

Text Segment	Code (fun(), main())
Data Segment	cnt = 100 (initialized global)
BSS Segment	Uninitialized globals
Stack/Heap	Local and dynamic data

At this stage, cnt is **physically allocated** and initialized.

---

(4) Run Time

Execution begins:

in main 100
in fun
100

---

4. Scope of a Variable

Local Scope

- Variable is visible only within the block/function it is defined in.

```
void disp() {
   int k = 10;
   printf("%d\n", k);
}
```

Here, k exists only within disp().

External (Global) Scope

Defined **outside all functions** — visible to all functions after its definition.

```
int var = 20;
void disp() {
   int k = 10;
   printf("%d\t%d\n", k, var);
}
```

The variable var is accessible throughout the file.

Global Variable Shadowing

If a **local variable** has the same name as a global variable,
the **local variable takes precedence** within that block.

```
int val = 10; // global
void main() {
  int val = 20; // local shadows global
  printf("%d\n", val);  // prints 20
}
```

Storage Classes

Storage classes define the **default value**, **memory location**, **scope**, and **lifetime** of a variable.

Attribute	Meaning
Default Initial Value	The value automatically assigned if not initialized
Memory Location	Segment where variable resides (stack/data/register)
Scope	Where variable can be accessed
Lifetime	Duration variable exists in memory

There are **four storage classes** in C/C++:

1. **Automatic (auto)**
2. **Static (static)**
3. **Register (register)**
4. **External (extern)**

---

1. Automatic Storage Class

Syntax:

```
auto int num;
or simply:
int num;
```

Characteristics:

- **Default initial value:** Garbage
- **Memory location:** Stack
- **Scope:** Local to block
- **Lifetime:** Exists until control exits the block

Example:

```
void disp() {
  int num;
}
```

2. Static Storage Class

Syntax:

static int var;

Characteristics:

- **Default initial value:** 0
- **Memory location:** Data Segment or BSS
- **Scope:** Local to the block in which it is defined
- **Lifetime:** Entire program execution

Example:

```
#include<stdio.h>
void disp() {
  int a = 0;
  static int b;
  printf("%d\t%d\n", a++, b++);
}
void main() {
  for (int i = 0; i < 3; i++)
    disp();
}
```

Output:

```
0  0
0  1
0  2
```

---

3. Register Storage Class

Syntax:

register int count;

Characteristics:

- **Default initial value:** Garbage
- **Memory location:** CPU register (if available)

- **Scope:** Local to block
- **Lifetime:** Until block execution ends

Notes:

- Provides **faster access** to frequently used variables.
- If CPU registers are full, compiler treats it as **automatic**.

---

4. External Storage Class

Syntax:

extern int var;

Purpose:
Used to access a variable **defined in another file** or **below its declaration** in the same file.

Example 1:

```
#include<stdio.h>
void disp();
extern int var;
int main() {
   disp();
   printf("%d\n", var);
}
int var = 200;
void disp() {
   printf("%d\n", var);
}
```

Example 2 (Multiple files):

```
// first.c
#include<stdio.h>
void fun() {
   extern int cnt;
   printf("in fun %d\n", cnt);
}

// second.c
#include "first.c"
int cnt = 100;
void main() {
   printf("in main %d\n", cnt);
   fun();
```

}

Characteristics:

- **Default initial value:** 0
- **Memory location:** Data Segment or BSS
- **Scope:** Global
- **Lifetime:** Until program termination

---

6. Variable Types Summary

Type	Declared In	Memory	Default Value	Scope	Lifetime
Automatic	Inside function	Stack	Garbage	Block	Until function ends
Static (local)	Inside function	Data/BSS	0	Block	Entire program
Static (global)	Outside function	Data/BSS	0	File	Entire program
Register	Inside function	CPU Register	Garbage	Block	Until function ends
Extern	Declared anywhere	Data/BSS	0	Global	Entire program

---

7. Local vs Global vs Static vs Extern

Aspect	Local	Global	Static	Extern
Defined Inside Function	Yes	No	Yes	No
Accessible Everywhere	No	Yes	File-level only	Yes
Memory Allocated	Stack	Data/BSS	Data/BSS	Data/BSS
Default Value	Garbage	0	0	0
Lifetime	Till function ends	Till program ends	Till program ends	Till program ends

---

Static Library in Visual Studio 2022

1. Objective

To create a **Static Library (.lib)** project in Visual Studio 2022, export a function from it, and then link this library with a **Client (Console Application)** project that uses the function.

---

2. Folder Structure

Create the following folders manually:

C:\MyCpp\developer
C : \MyCpp\client
C : \forclient

. Developer Project (Static Library)

Step 1: Create a Static Library Project

1. Open **Visual Studio 2022**.
2. Go to **File → Close Solution** (if any project is open).
3. Select **Create a new project**.
4. Choose **Static Library** (C++).
5. Click **Next**.
6. Enter:
    - **Project Name:** devpro
    - **Location:** C:\MyCpp\developer
7. Check **Place solution and project in the same directory**.
8. Click **Create**.

Step 2: Configure Precompiled Header (pch.h)

Open the generated file **pch.h** and ensure it contains:

```
#ifndef PCH_H
#define PCH_H

// add headers that you want to pre-compile here
#include "framework.h"
#include <iostream>
using namespace std;

#endif //PCH_H
```

Step 3: Add a Header File

1. Right-click on **devpro** → **Add** → **New Item**.
2. Select **Header File (.h)**.
3. Name it **myheader.h**.
4. Type and save:

```
void disp();
```

Step 4: Define the Function

Open **devpro.cpp** and replace its contents with:

```
#include "pch.h"
#include "myheader.h"

void disp()
{
   cout << "Welcome to static library function" << endl;
}
```

Step 5: Build the Static Library

1. Compile: **Ctrl + F7**
2. Build: **Ctrl + Shift + B**

After a successful build, you will find the generated static library file at:

C:\MyCpp\developer\devpro\x64\Debug\devpro.lib

Step 6: Share the Library

Copy the following two files to the shared folder:

C:\forclient\myheader.h
C:\forclient\devpro.lib

---

4. Client Project (Console Application)

Step 1: Create Client Project

1. Open **Visual Studio 2022** (new instance).
2. Select **Create a new project**.
3. Choose **Console App (C++)**.
4. Click **Next**.
5. Enter:

- **Project Name:** clientpro
- **Location:** C:\MyCpp\client

6. Check **Place solution and project in the same directory**.
7. Click **Create**.

Step 2: Modify the Code

Replace the contents of **clientpro.cpp** with:

```
#include <iostream>
#include "C:\forclient\myheader.h"
using namespace std;

int main()
{
  cout << "Before Calling" << endl;
  disp();
  cout << "After Calling" << endl;
}
```

Step 3: Compile the Client

1. Save the file.
2. Compile (Ctrl + F7).
   - This will compile successfully (no syntax errors).
   - However, building will fail with a **linker error** because the body of disp() is missing.

Step 4: Link with the Static Library

1. Right-click on **clientpro** → **Properties**.
2. Navigate to:
3. Configuration Properties → Linker → Input
4. On the right side, select the field **Additional Dependencies**.
5. Click the **Edit** button.
6. Add the following path:
7. C:\forclient\devpro.lib
8. Click **Apply → OK**.

Step 5: Build and Run

1. Build (Ctrl + Shift + B).

- This time, the linker will successfully find the disp() function in devpro.lib.

2. Run (Ctrl + F5).

Output:

Before Calling
Welcome to static library function
After Calling

---

5. Conceptual Understanding

Static Library (.lib)

- A **static library** is a collection of precompiled object files that can be linked directly into an executable at **link time**.
- Once linked, the code becomes part of the final executable (.exe).
- No separate .lib file is required at runtime.

Precompiled Header (pch.h)

- The pch.h file improves **compilation speed**.
- It contains headers that are common to many source files (e.g., <iostream>).
- Instead of parsing these headers for every file, the compiler parses them once and reuses the precompiled data.

---

6. Summary Table

Step	Action	Output File / Effect
1	Create Static Library (devpro)	Generates devpro.lib
2	Create Header (myheader.h)	Function declaration
3	Implement Function (disp())	Function definition
4	Copy .lib and .h to shared folder	C:\forclient
5	Create Client Project (clientpro)	Uses disp() function
6	Include header + link library	Successful build
7	Run	Displays “Welcome to static library function”

Creating a Dynamic-Link Library (DLL) in C++

Step 1: Folder Setup

Create the following folder structure:

```
C:\MyCPP
  ├── developer
  └── client
```

Step 2: Create a New DLL Project

In Visual Studio 2022:

1. Go to **File → New → Project**
2. Select **Dynamic-Link Library (DLL)** project.
3. Enter:
    - **Name:** devpro
    - **Location:** C:\MyCPP\developer\
4. Select **Place project and solution in the same directory.**
5. Click **Create**.

Step 3: Edit the Precompiled Header (pch.h)

In the generated **pch.h** file, ensure the following content:

```
#ifndef PCH_H
#define PCH_H

// add headers that you want to pre-compile here
#include "framework.h"
#include <iostream>
using namespace std;

#endif // PCH_H
```

Step 4: Create Header File

1. Right-click on the project name **devpro**
2. Select **Add → New Item → Header File (.h)**
3. Name it **myheader.h**
4. Add the following code:

```
extern "C" __declspec(dllimport) void disp2();
```

Explanation:

- extern "C" prevents C++ name mangling, allowing compatibility across languages.
- __declspec(dllimport) tells the compiler that this function's implementation resides in an external DLL.

Step 5: Implement Function in the DLL Source

In the generated **dllmain.cpp**, write:

```
#include "pch.h"

extern "C" __declspec(dllexport) void disp2()
{
   cout << "Welcome to disp2 function" << endl;
}
```

Explanation:

- __declspec(dllexport) exposes this function from the DLL to be used by other programs.
- extern "C" ensures the exported function uses standard C naming (important for linking).

Step 6: Build the DLL

1. Compile the project: **Ctrl + F7**
2. Build the solution: **Ctrl + Shift + B**

Ensure that the following files are created in:

C:\MyCPP\developer\devpro\x64\Debug

Files generated:

- devpro.dll → Dynamic Library File
- devpro.lib → Import Library
- myheader.h → Header file for client use

Step 7: Prepare Files for Client

Copy the following files to a shared folder:

```
C:\forclient
   ├── devpro.dll
   ├── devpro.lib
```

└── myheader.h

Step 8: Create a Client Application

In a new instance of Visual Studio:

1. Go to **File → New → Project**
2. Select **Console App**
3. Enter:
    - **Name:** clientpro
    - **Location:** C:\MyCPP\client\
4. Select **Place solution and project in the same directory.**
5. Click **Create**

Step 9: Modify Client Code

In the generated **clientpro.cpp**, write:

```
#include <iostream>
using namespace std;

int main()
{
   cout << "Before Calling" << endl;
   disp2();
   cout << "After Calling" << endl;
}
```

If you compile now, you will get the error:

Identifier “disp2” is undefined.

Add the header file:

```
#include "C:\forclient\myheader.h"
```

Now compile (**Ctrl + F7**) → It will compile successfully.
Build (**Ctrl + Shift + B**) → It will show **linker error** because the linker cannot find the body of disp2.

Step 10: Configure Linker Settings

To resolve the linker error:

1. Right-click on **clientpro → Properties**
2. Go to:

3. Configuration Properties
4. → Linker
5. → Input
6. → Additional Dependencies
7. Click **Edit**, and type:
8. C:\forclient\devpro.lib
9. Click **Apply → OK**

Now **build** the project (**Ctrl + Shift + B**).
It will successfully build and create clientpro.exe.

Step 11: Resolve DLL Not Found Error

If you run (**Ctrl + F5**) and get:

devpro.dll not found

Copy devpro.dll from:

C:\forclient\

to:

C:\MyCPP\client\clientpro\x64\Debug

Now run again (**Ctrl + F5**) →
Output:

Before Calling
Welcome to disp2 function
After Calling

Concept of Import Library

The devpro.lib file contains the **information the linker needs** to resolve external references to functions exported by the DLL. At runtime, the system uses this information to **locate and load the DLL** and its exported functions.

Understanding dllexport and dllimport

When Creating a DLL (Developer Side)

Use:

__declspec(dllexport) void disp2();

This tells the compiler to **export** the function and make it accessible to other programs.

When Using a DLL (Client Side)

Use:

__declspec(dllimport) void disp2();

This tells the compiler that the **implementation exists in an external DLL** and must be linked at runtime.

Static Library (.lib) vs Dynamic Library (.dll)

Aspect	Static Library (.lib)	Dynamic Library (.dll)
Linking Time	Linked at **compile time**	Linked at **runtime**
Integration	Code from .lib becomes part of .exe	.exe loads DLL functions dynamically
File Needed at Runtime	Only .exe needed	.dll must be present
Executable Size	Larger	Smaller
Update Process	Must recompile app after library change	Can replace DLL independently
Sharing Between Apps	Not shared; each .exe has its own copy	Shared among multiple programs
Performance	Slightly faster execution	Slightly slower load time due to runtime linking

---

Role of pch.h (Precompiled Header)

- pch.h stands for **Precompiled Header**.
- It speeds up compilation by precompiling commonly used headers (like <iostream>, <string>, etc.).
- Once compiled, these headers are not re-parsed for each source file, improving build performance.

Java JNI and DLL Linking (Advanced Integration)

If the DLL is to be used in **Java**, the function must follow **JNI naming conventions**.

Expected Function Signature

For a Java class:

```
package mypack;
public class TestDll {
   public native void disp2();
}
Java expects a native function in C++ :
   extern "C" JNIEXPORT void JNICALL Java_mypack_TestDll_disp2(JNIEnv*, jobject)
{
   cout << "Welcome to the dynamic library!" << endl;
}
```

Key Points

- Use javac -h . mypack/TestDll.java to generate the JNI header file.
- Match the package, class, and method names exactly.
- Compile the DLL and ensure it is accessible through:
- System.load("C:\\FullPath\\devpro.dll");